```
"""
RAG System - Retrieval and Generation
Builds on the indexing pipeline you already have
"""

# IMPORTS

import os
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings, ChatOpenAI


# ============================================================
# RETRIEVAL COMPONENT
# ============================================================

def retrieve_documents(query, vectorstore, k=3):
    """
    Retrieve most relevant document chunks for a query

    Args:
        query: User's question
        vectorstore: Vector database instance
        k: Number of chunks to retrieve

    Returns:
        List of relevant document chunks with scores
    """

    # Perform similarity search with scores
    results = vectorstore.similarity_search_with_score(query,k=k)

    # Format results
  retrieved_chunks = []

    for doc, score in results:
        retrieved_chunks.append({
            'content': doc.page_content,
            'score': score,
            'metadata': doc.metadata
```

```
        })

    print(f"✅ Retrieved {len(retrieved_chunks)} relevant chunks")
    return retrieved_chunks

# ============================================================
# PROMPT ENGINEERING
# ============================================================

def build_rag_prompt(query, retrieved_chunks):
    """
    Construct prompt with context and query

    Args:
        query: User's question
        retrieved_chunks: List of retrieved document chunks

    Returns:
        Formatted prompt string
    """

    # Combine all retrieved chunks into context
    context = "\n\n---\n\n".join(
        [chunk['content'] for chunk in retrieved_chunks]
    )

    # Build structured prompt
    prompt = f"""You are a helpful assistant that answers questions based
on the provided context.

INSTRUCTIONS:
- Use ONLY the information from the context below to answer the question
- If the answer is not in the context, say "I don't have enough
information to answer that question."
- Be concise and accurate
- Cite specific parts of the context when relevant

CONTEXT: {context}

QUESTION: {query}
```

```
ANSWER:
"""

    return prompt



# ============================================================
# GENERATION COMPONENT
# ============================================================

def query_rag_system(question, vectorstore, k=3, verbose=True):
    """
    Complete RAG query: Retrieve + Generate

    Args:
        question: User's question
        vectorstore: Vector database instance
        k: Number of chunks to retrieve
        verbose: Print detailed information

    Returns:
        Dictionary with answer and metadata
    """

    print("\n" + "=" * 70)
    print("RAG QUERY PIPELINE")
    print("=" * 70)

    # Step 1: Retrieve relevant chunks
    retrieved_chunks = retrieve_relevant_chunks(
        question,
        vectorstore,
        k=k
    )

    if verbose:
        print("\n[RETRIEVED CHUNKS]")
```

```
        for i, chunk in enumerate(retrieved_chunks, 1):
            print(f"\nChunk {i} (Score: {chunk['score']:.4f}):")
            print(chunk['content'][:200] + "...")

    # Step 2: Build prompt
    rag_prompt = build_rag_prompt(
        question,
        retrieved_chunks
    )

    if verbose:
        print(f"\n[PROMPT] Length: {len(rag_prompt)} characters")

    # Step 3: Generate answer
    answer = generate_response(rag_prompt)

    # Return results
    return {
        'question': question,
        'answer': answer,
        'retrieved_chunks': retrieved_chunks,
        'num_chunks_used': len(retrieved_chunks)
    }

# ============================================================
# EXAMPLE USAGE
# ============================================================
if __name__ == "__main__":
    print("=" * 70)
    print("RAG QUERY SYSTEM - Manual Pipeline")
    print("=" * 70)

    # Load existing vector store
    print("\n[SETUP] Loading vector store...")

    embeddings = HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    )

    vectorstore = Chroma(
```

```
        persist_directory="./chroma_db",
        embedding_function=embeddings
    )

    print("✅ Vector store loaded")

    # Example queries
    queries = [
        "What is this document about?",
        "Tell me about the yellow character",
        "What are the main topics covered?"
    ]

    for query in queries:
        result = query_rag_system(
            query,
            vectorstore,
            k=3,
            verbose=True
        )

        print("\n" + "=" * 70)
        print("FINAL ANSWER")
        print("=" * 70)

        print(f"Q: {result['question']}")
        print(f"A: {result['answer']}")
        print(f"Chunks used: {result['num_chunks_used']}")
        print("=" * 70)

    print("\n✅ RAG system ready for interactive queries!")
    print("\nTo use interactively:")
    print("result = query_rag_system('Your question here', vectorstore)")
```